

OICS Evidentiary Anchoring Specification v1.0

Status: Frozen Candidate (Ready to hash / seal)

Owner / Steward: Virgotech Holdings LLC (initial steward)

Author: Eric J. Turk

0. Purpose and Scope

0.1 Purpose

OICS defines a deterministic method to create **verifiable evidentiary records** (“OICS Events”) that prove:

- an event occurred,
- the event content has not changed,
- the event can be independently verified,
- optional external anchoring can provide additional timestamp attestability.

0.2 Scope

OICS applies to any “event” where integrity, provenance, and later verification matter: governance changes, contract deployments, document finalization, configuration updates, disclosures, attestations, etc.

0.3 What OICS is NOT

OICS does **not**:

- replace an authoritative ledger of record,
- dictate settlement finality,
- require a specific blockchain,
- expose underlying confidential content (OICS supports hash-only proofs), function as a consensus network.

1. Terms

- **Event:** A discrete occurrence represented in OICS as a structured record.
- **Event Payload:** The canonicalized bytes derived from an Event's structured representation.
- **Event Hash:** Cryptographic digest of the Event Payload.
- **Anchor:** Optional publication of an Event Hash (or Merkle root) into an external timestamped system.
- **Evidence Bundle:** The complete set of files needed to verify an event (event JSON, attachments, hashes, manifests, logs).
- **Canonicalization:** A deterministic method to produce identical bytes from identical logical content.

2. Event Definition Model

2.1 Event Object

An OICS Event MUST be represented as a JSON object with the following top-level fields:

- `oics_version` (string) — MUST be "1.0"
- `event_id` (string) — unique identifier for the event (see 2.2)
- `event_type` (string) — classification (see 2.3)
- `created_at` (string) — RFC 3339 timestamp in UTC (see 3.4)
- `issuer` (object) — identity claims (see 2.4)
- `subject` (object) — what the event is about (see 2.5)
- `artifacts` (array) — referenced files/attachments with hashes (see 2.6)
- `claims` (object) — human-readable assertions, optional but recommended (see 2.7)
- `context` (object) — optional supporting context

- `signatures` (array) — optional cryptographic signatures (see 6.3)
- `anchors` (array) — optional anchoring proofs (see 5)

2.2 `event_id` format

`event_id` MUST be generated deterministically or randomly but must be unique.
Allowed formats:

Option A (recommended): UUIDv4 string

Option B: `OICS-YYYYMMDD-HHMMSSZ-<8 hex>` (issuer-chosen)

Example: `OICS-20260227-173012Z-9f3a2c10`

2.3 `event_type` registry

The following event types are RECOMMENDED (extendable):

- `DOC_FREEZE` — document declared frozen / finalized
- `SEAL_ISSUED` — seal applied / mark of receipt / issuance
- `GOV_CHANGE` — governance / policy change event
- `CONFIG_CHANGE` — configuration update
- `CODE_DEPLOY` — deployment or release event
- `ATTESTATION` — statement by issuer about a fact
- `DISCLOSURE` — disclosure of information to entitled party
- `TRANSFER` — custody/ownership assertion (hash-anchored)
- `OTHER` — explicitly described in `claims`

2.4 `issuer` object

MUST contain:

- `name` (string)
- `entity_type` (string) — e.g., `INDIVIDUAL`, `LLC`, `CORP`, `FOUNDATION`
- `jurisdiction` (string) — e.g., `US-GA`

- `identifier` (object, optional) — e.g., registration/EIN later
- `contact` (object, optional)

2.5 subject object

MUST contain:

- `title` (string)
- `description` (string, optional)
- `domain` (string, optional) — e.g., OICS, BTP, CANTON_INTEGRATION
- `tags` (array of strings, optional)

2.6 artifacts array

Each artifact entry MUST contain:

- `name` (string) — filename
- `media_type` (string) — MIME type (e.g., application/pdf)
- `sha256` (string) — lowercase hex
- `size_bytes` (integer, optional but recommended)
- `role` (string, optional) — e.g., PRIMARY_DOC, SEAL_IMAGE, LOG, MANIFEST

Rule: If the event references a document, the document MUST appear here with its hash.

2.7 claims object

Recommended fields:

- `statement` (string) — plain language statement of what is being asserted
- `intent` (string, optional) — why the event exists
- `notes` (string, optional)

3. Canonical Data Formatting Rules

3.1 Character encoding

All string content **MUST** be UTF-8.

3.2 Canonical JSON (OICS-JCS-1.0)

To produce the Event Payload, the Event JSON **MUST** be canonicalized as follows:

1. JSON object keys **MUST** be sorted lexicographically (byte-order, UTF-8).
2. No insignificant whitespace (no pretty printing).
3. Numbers **MUST** be represented without trailing zeros (use JSON normal form).
4. Arrays preserve order exactly as provided.
5. Strings **MUST** use JSON escaping rules; no alternative unicode escaping unless required.
6. The canonicalized output **MUST** be encoded as UTF-8 bytes.

(If you want a named standard reference later, we can align this to RFC 8785-style JCS behavior; for v1.0 the above rules are explicit and implementable.)

3.3 Field inclusion rules

- Fields with `null` values **MUST** be omitted.
- Optional fields not used **MUST** be omitted (not empty).

3.4 Timestamp format

`created_at` **MUST** be RFC 3339 UTC with `Z`.

Example: `2026-02-27T17:30:12Z`

4. Hashing Algorithm

4.1 Primary hash

Event Hash **MUST** be:

- Algorithm: **SHA-256**

- Input: UTF-8 bytes of the canonicalized Event JSON (Event Payload)
- Output: lowercase hex string (64 hex chars)

4.2 Artifact hashes

All artifacts **MUST** use SHA-256 of raw file bytes, output as lowercase hex.

4.3 Optional Merkle bundling

If anchoring many events at once, issuers **MAY** compute:

- a Merkle tree over Event Hashes (sorted lexicographically),
- anchor the Merkle root,
- include the Merkle proof for each event in its `anchors` entry.

5. Optional Anchoring Procedure

Anchoring is **OPTIONAL**. OICS remains valid without anchoring.

5.1 Anchor record structure

Each `anchors[ ]` entry **SHOULD** include:

- `anchor_type` (string) — e.g., `PUBLIC_CHAIN_TX`, `TIMESTAMP_AUTHORITY`, `NOTARY`, `INTERNAL_LEDGER`
- `anchored_hash` (string) — Event Hash or Merkle root
- `anchored_at` (string) — RFC3339 UTC time of anchor observation
- `reference` (object) — depends on `anchor_type` (see below)
- `proof` (object, optional) — Merkle path, receipt, etc.

5.2 Recommended anchor types

- `PUBLIC_CHAIN_TX`: includes { `"chain": "...", "txid": "...", "block_height": n` }

- `TIMESTAMP_AUTHORITY`: includes { `"provider": "...", "receipt_id": "..."` }
- `NOTARY`: includes { `"notary": "...", "jurisdiction": "...", "record_id": "..."` }
- `INTERNAL_LEDGER`: includes { `"system": "...", "record_pointer": "..."` }

Rule: Anchoring MUST NOT require revealing underlying artifacts. Only hashes are anchored unless the issuer chooses otherwise.

6. Verification Procedure

A verifier confirms an OICS Event using an Evidence Bundle.

6.1 Minimum Evidence Bundle

A valid bundle MUST contain:

- `event.json` (the OICS Event)
- `manifest.json` (recommended; see 7)
- all referenced artifacts (or a statement that they are withheld, if hash-only verification is intended)
- hash logs (optional but recommended)

6.2 Verification steps

1. Parse `event.json`.
2. Ensure `oics_version == "1.0"`.
3. Confirm required fields exist (Section 2.1).
4. Recompute SHA-256 for each artifact file and match `artifacts[ ].sha256`.
5. Canonicalize `event.json` per Section 3.
6. Compute Event Hash per Section 4.1.

7. If anchors exist:

- verify anchor references (txid/receipt/etc.)
- confirm anchored hash equals Event Hash or Merkle root
- if Merkle root, verify Merkle proof binds Event Hash to root.

6.3 Optional signatures

If `signatures[ ]` exists, each signature entry SHOULD include:

- `sig_type` (e.g., ED25519, SECP256K1)
- `public_key` (base64 or hex)
- `signature` (base64 or hex)
- `signed_hash` (Event Hash)
- `signed_at` (RFC3339 UTC)

Verification: validate signature over the Event Hash.

(If you're not ready to commit to signature schemes, keep this optional. Institutions like optional signatures.)

7. Metadata Structure

7.1 Manifest (recommended)

Include `manifest.json` describing the bundle contents.

Fields:

- `bundle_version`: "1.0"
- `bundle_id`: string
- `created_at`: RFC3339 UTC
- `files`: array of { "name", "sha256", "size_bytes", "role" }
- `event_hash`: Event Hash (computed)

- notes: optional

This prevents “missing file” confusion.

7.2 Human-readable README (recommended)

Include `README.txt` describing:

- what this bundle is,
- how to verify,
- where to find hashes,
- any withheld artifacts.

8. Indexing Schema

OICS indexing is deterministic and supports later retrieval.

8.1 Required index keys

An implementation SHOULD index on:

- `event_id`
- `event_type`
- `created_at`
- `issuer.name`
- `subject.domain`
- `each artifacts[ ].sha256`

8.2 Suggested index string (portable)

Create a stable index line format:

```
OICS|1.0|<created_at>|<event_type>|<issuer_name>|  
<event_id>|<event_hash>
```

This can live in:

- a ledger file
- an internal database
- a published index

9. Conflict Handling Rules

Conflicts happen. OICS defines how to reason about them.

9.1 Same event_id, different event_hash

If two events share `event_id` but produce different Event Hash:

- treat as **conflict**
- the later one **MUST** use a new `event_id`
- issuers **SHOULD** issue a `CONFLICT_NOTICE` event that references both hashes in `artifacts` or `context`.

9.2 Same artifacts, different claims

If artifacts hashes match but claims differ:

- artifacts integrity is intact
- claims divergence is a provenance issue
- use signatures and anchors to determine which claim set is authoritative for a given domain.

9.3 Anchor conflicts

If anchors disagree (e.g., multiple timestamps):

- anchors are additive evidence
- earliest verifiable anchor timestamp generally carries stronger “existence by” proof

- authoritative record remains domain-specific (OICS does not override a system of record).

10. Versioning and Upgrade Policy

10.1 Semantic versioning

OICS versions are MAJOR.MINOR.

- **MAJOR** change: breaks canonicalization or required fields (invalidates old verifiers)
- **MINOR** change: adds optional fields or new event types, backward compatible

10.2 Compatibility promise

- OICS 1.x verifiers **MUST** accept events with `oics_version: "1.0"` and ignore unknown optional fields.
- Canonicalization rules **MUST NOT** change within 1.x.

10.3 Deprecation

Deprecated fields **MUST** remain verifiable for at least one minor version cycle.

11. Governance Statement

11.1 Stewardship (v1.0)

Initial stewardship is held by **Virgotech Holdings LLC**, responsible for:

- publishing the canonical text of OICS v1.x
- maintaining a public changelog
- issuing cryptographic hashes for released specs

11.2 Change control (v1.x)

Changes to OICS within v1.x:

- **MUST** be documented in a changelog

- MUST be published with hash of the updated specification
- SHOULD include an accompanying SPEC_CHANGE OICS Event anchoring the update

11.3 Neutrality

OICS is designed to be implementable by any party. Implementations MAY be proprietary, but the **verification rules** MUST remain consistent with this specification to claim OICS compliance.

(If you want to go “open standard” later, we can add a foundation pathway. For now, this governance statement is clean and non-fiction.)

12. Reference Implementation Guidance

An OICS-compliant implementation MUST provide:

- a function to canonicalize event JSON (Section 3)
- SHA-256 hashing (Section 4)
- artifact hashing and validation (Section 4.2)
- verification routine (Section 6)

OPTIONAL:

- signature support
- anchor verification support
- Merkle batching support

Appendix A — Minimal Event Example (v1.0)

```
{
  "oics_version": "1.0",
  "event_id": "OICS-20260227-173012Z-9f3a2c10",
  "event_type": "DOC_FREEZE",
  "created_at": "2026-02-27T17:30:12Z",
```

```
"issuer": {
  "name": "Virgotech Holdings LLC",
  "entity_type": "LLC",
  "jurisdiction": "US-GA"
},
"subject": {
  "title": "OICS Evidentiary Anchoring Specification
v1.0",
  "domain": "OICS",
  "tags": ["spec", "freeze", "v1.0"]
},
"artifacts": [
  {
    "name": "OICS_Evidentiary_Anchoring_Spec_v1.0.pdf",
    "media_type": "application/pdf",
    "sha256": "<INSERT_SHA256>",
    "role": "PRIMARY_DOC"
  }
],
"claims": {
  "statement": "This document is declared frozen as OICS
Evidentiary Anchoring Specification v1.0."
}
}
```